// Implementing stack using arrays

```c
#include <stdio.h>
#include <stdlib.h>

void push();
void pop();
void display();
void search();

int stack[100], top=-1, num;

int main() {
    printf("\nEnter the size of the stack: ");
    scanf("%d", &num);

    int ch;
    while(1) {
        printf("\nStack Operations: ");
        printf("\n1. push\n2. pop\n3. display\n4. search\n5. exit");
        printf("\nEnter your choice: ");
        scanf("%d", &ch);

        switch(ch) {
            case 1:
                push();
                break;
            case 2:
                pop();
                break;
            case 3:
                display();
                break;
            case 4:
                search();
                break;
            case 5:
                printf("\nExited\n");
                exit(0);
                break;
            default:
                printf("\nInvalid choice, try again.\n");
        }
    }
    return 0;
}

void push() {
    if(top == (num-1))
        printf("\nStack is full\n");
    else {
        int x;
        printf("\nEnter value: ");
        scanf("%d", &x);
        top++;
        stack[top] = x;
        printf("\nPushed\n");
    }
}

void pop() {
    if(top == 0)
        printf("\nStack is empty\n");
    else {
        int x;
        x = stack[top];
        top--;
        printf("\nPoped %d\n", x);
    }
}

void display() {
    if(top == 0) {
        printf("\nThere are no elements in the stack\n");
        return;
    }
    printf("\nElements in the stack are: \n");
    for(int i=0; i<num; i++) {
        if(i <= top)
            printf("%d ", stack[i]);
        else
            printf("- ");
    }
    printf("\n");
}

void search() {
    if(top == 0) {
        printf("\nThere are no elements in the stack\n");
        return;
    }
    int key;
    printf("\nEnter the key value: ");
    scanf("%d", &key);
    int flag=1;
    for(int i=0; i<=top; i++) {
        if(stack[i] == key) {
            printf("Found at position: %d\n", i+1);
            flag = 0;
```

```
        }
    }
    if(flag)
        printf("Key not found\n");
}
```

// Implementing queue using arrays

```
#include <stdio.h>
#include <stdlib.h>

void insert();
void dele();
void display();
void search();

int queue[100], rear=-1, front=-1, num;

int main() {
    printf("\nEnter the size of the queue: ");
    scanf("%d", &num);

    int ch;
    while(1) {
        printf("\nQueue Operations: ");
        printf("\n1. insert\n2. delete\n3. display\n4. search\n5.
exit");
        printf("\nEnter your choice: ");
        scanf("%d", &ch);

        switch(ch) {
            case 1:
                insert();
                break;
            case 2:
                dele();
                break;
            case 3:
                display();
                break;
            case 4:
                search();
                break;
            case 5:
                printf("\nExited\n");
                exit(0);
                break;
            default:
                printf("\nInvalid choice, try again.\n");
        }
    }
    return 0;
}

void insert() {
```

```
    if(rear == (num-1)) {
        printf("\nQueue is full\n");
        return;
    }

    int x;
    printf("\nEnter value: ");
    scanf("%d", &x);
    rear++;
    queue[rear] = x;
    printf("Inserted\n");
}

void dele() {
    if(rear == -1) {
        printf("\nQueue is empty, cannot delete element\n");
        return;
    }

    front++;
    int x = queue[front];
    if(front == rear)
        front = rear = -1;
    printf("\nDeleted %d\n", x);
}

void display() {
    if(rear == -1) {
        printf("\nThere are no elements in the queue\n");
        return;
    }

    printf("\nThe elements in the queue are: \n");
    for(int i=0; i<=rear; i++) {
        if(i<=front)
            printf("- ");
        else
            printf("%d ", queue[i]);
    }
    printf("\n");
}

void search() {
    if(rear == -1) {
        printf("\nThere are no elements in the queue\n");
        return;
    }
    int key;
    printf("\nEnter the key value: ");
    scanf("%d", &key);
    int flag=1;
    for(int i=front+1; i<=rear; i++) {
        if(queue[i] == key) {
            printf("Found at position: %d\n", i+1);
            flag = 0;
        }
    }
    if(flag)
        printf("Key not found in the queue\n");
}
```

```c
// Implement stack using linked list

#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node * link;
}Node;

Node * top = NULL;

void push();
void pop();
void display();
void search();

int main() {
    int ch;
    while(1) {
        printf("\nStack Operations: ");
        printf("\n1. push\n2. pop\n3. display\n4. search\n5. exit");
        printf("\nEnter your choice: ");
        scanf("%d", &ch);

        switch(ch) {
            case 1:
                push();
                break;
            case 2:
                pop();
                break;
            case 3:
                display();
                break;
            case 4:
                search();
                break;
            case 5:
                printf("\nExited\n");
                exit(0);
                break;
            default:
                printf("\nInvalid choice, try again.\n");
        }
    }
    return 0;
}

void push() {
    Node * new = (Node *)malloc(sizeof(Node));
    printf("\nEnter the element: ");
    scanf("%d", &new->data);
    new->link = top;
    top = new;
    printf("Pushed\n");
}

void pop() {
    if(top == NULL) {
        printf("\nStack is empty, cannot perform pop operation.\n");
        return;
    }

    Node * temp = top;
    top = top->link;
    printf("\nPoped %d\n", temp->data);
    free(temp);
}

void display() {
    if(top == NULL) {
        printf("\nNo elements in the stack\n");
        return;
    }

    Node * temp = top;
    printf("\nElements in the stack are: \n");
    printf("top");
    while(temp != NULL) {
        printf(" --> %d", temp->data);
        temp = temp->link;
    }
    printf("\n");
}

void search() {
    if(top==NULL) {
        printf("\nNo elements in the stack\n");
        return;
    }

    int key;
    printf("\nEnter the key: ");
    scanf("%d", &key);
    Node * temp = top;
    int flag=1, pos=1;
    while(temp != NULL) {
        if(temp->data == key) {
```

```
            printf("Found at position: %d\n", pos);
            flag=0;
        }
        temp = temp->link;
        pos++;
    }
    if(flag)
        printf("Key not found\n");
}
```

// Implement queue using linked list

```
#include <stdio.h>
#include <stdlib.h>

typedef struct node {
    int data;
    struct node * link;
}Node;

Node * rear = NULL;
Node * front = NULL;

void insert();
void dele();
void display();
void search();

int main() {
    int ch;
    while(1) {
        printf("\nQueue Operations: ");
        printf("\n1. insert\n2. delete\n3. display\n4. search\n5.
exit");
        printf("\nEnter your choice: ");
        scanf("%d", &ch);

        switch(ch) {
            case 1:
                insert();
                break;
            case 2:
                dele();
                break;
            case 3:
                display();
                break;
            case 4:
                search();
                break;
            case 5:
                printf("\nExited\n");
                exit(0);
                break;
            default:
                printf("\nInvalid choice, try again.\n");
        }
    }
    return 0;
```

```
}

void insert() {
    Node * new = (Node *)malloc(sizeof(Node));
    printf("\nEnter value: ");
    scanf("%d", &new->data);
    new->link = NULL;
    if(rear != NULL)
        rear->link = new;
    rear = new;
    if(front == NULL)
        front = new;
    printf("Inserted\n", new->data);
}

void dele() {
    if(rear == NULL) {
        printf("\nQueue is empty\n");
        return;
    }

    Node * temp = front;
    front = front->link;
    if(front == NULL)
        rear = NULL;
    printf("\nDeleted %d\n", temp->data);
    free(temp);
}

void display() {
    if(rear == NULL) {
        printf("\nNo elements in the queue\n");
        return;
    }

    Node * temp = front;
    printf("\nElements in the queue are: \n");
    printf("front --> ");
    while(temp != NULL) {
        printf("%d --> ", temp->data);
        temp = temp->link;
    }
    printf("rear");
    printf("\n");
}

void search() {
    if(rear==NULL) {
        printf("\nNo elements in the queue\n");
        return;
    }

    int key;
    printf("\nEnter the key: ");
    scanf("%d", &key);
    Node * temp = front;
    int flag=1, pos=1;
    while(temp != NULL) {
        if(temp->data == key) {
            printf("Found at position: %d\n", pos);
            flag=0;
        }
        temp = temp->link;
        pos++;
    }
    if(flag)
        printf("Key not found\n");
}
```

```c
// Implementing linked list

#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node * link;
};

struct node * head = NULL;
int count = 0;

void insertAtBegin(int x);
void insertAtEnd(int x);
void insertAtPosition(int x, int pos);
void deleteAtBegin();
void deleteAtEnd();
void deleteAtPosition(int pos);
void display();
void search(int key);

int main() {
    insertAtBegin(10);
    insertAtBegin(20);
    insertAtBegin(30);
    display();

    insertAtEnd(40);
    insertAtEnd(50);
    insertAtEnd(60);
    display();

    insertAtPosition(35, 2);
    insertAtPosition(0, 5);
    insertAtPosition(65, 7);
    display();

    search(40);
    search(75);

    deleteAtBegin();
    deleteAtBegin();
    display();

    deleteAtEnd();
    deleteAtEnd();
    display();

    deleteAtPosition(4);
    deleteAtPosition(2);
    display();

    return 0;
}

void insertAtBegin(int x) {
    struct node * new;
    new = (struct node * )malloc(sizeof(struct node));
    new->data = x;
    new->link = head;
    head = new;
    printf("\nInserted %d at the beginning", x);
    count++;
}

void insertAtEnd(int x) {
    if(head == NULL)
        insertAtBegin(x);
    else {
        struct node * temp = head;
        while(temp->link != NULL) {
            temp = temp->link;
        }
        struct node * new;
        new = (struct node * )malloc(sizeof(struct node));
        new->data = x;
        new->link = NULL;
        temp->link = new;
        printf("\nInserted %d at the end", x);
        count++;
    }
}

void insertAtPosition(int x, int pos) {
    if(pos<=0 || pos>(count+1)) {
        printf("Invalid position, insertion not possible");
        return;
    }
    if(pos == 1)
        insertAtBegin(x);
    else if(pos == (count+1))
        insertAtEnd(x);
    else {
        struct node * temp = head;
        for(int i=1; i<(pos-1); i++)
            temp = temp->link;
        struct node * new;
        new = (struct node * )malloc(sizeof(struct node));
```

```
        new->data = x;
        new->link = temp->link;
        temp->link = new;
        printf("\nInserted %d at position %d", x, pos);
        count++;
    }
}

void deleteAtBegin() {
    if(head==NULL) {
        printf("\nLinked list is empty");
        return;
    }

    struct node * temp = head;
    head = temp->link;
    printf("\nDeleted %d from the beginning", temp->data);
    free(temp);
    count--;
}

void deleteAtEnd() {
    if(head==NULL) {
        printf("\nLinked list is empty");
        return;
    }

    if(head->link == NULL) {
        deleteAtBegin();
    } else {
        struct node * prev = head;
        struct node * temp = head->link;
        while(temp->link != NULL) {
            temp = temp->link;
            prev = prev->link;
        }
        prev->link = NULL;
        printf("\nDeleted %d from the end", temp->data);
        free(temp);
        count--;
    }
}

void deleteAtPosition(int pos) {
    if(head==NULL) {
        printf("\nLinked list is empty");
        return;
    }

    if(pos<=0 || pos>count) {
        printf("Invalid position, deletion not possible");
        return;
    }
    if(pos == 1)
        deleteAtBegin();
    else if(pos == count)
        deleteAtEnd();
    else {
        struct node * prev = head;
        struct node * temp = head->link;
        for(int i=2; i<pos; i++) {
            temp = temp->link;
            prev = prev->link;
        }
        prev->link = temp->link;
        printf("\nDeleted %d from position %d", temp->data, pos);
        free(temp);
        count--;
    }
}

void display() {
    if(head == NULL) {
        printf("\nNo elements in the linked list");
        return;
    }

    struct node * temp = head;
    printf("\nElements in the linked list are: \n");
    printf("head");
    while(temp != NULL) {
        printf(" --> %d", temp->data);
        temp = temp->link;
    }
    printf("\n");
}

void search(int key) {
    if(head == NULL) {
        printf("\nNo elements in the linked list");
        return;
    }

    struct node * temp = head;
    int flag=1, pos=1;
    printf("\nSearching %d: ", key);
    while(temp != NULL) {
        if(temp->data == key) {
            printf("\nFound at position: %d", pos);
            flag=0;
```

```
        }
        temp = temp->link;
        pos++;
    }
    if(flag)
        printf("Key not found");
    printf("\n");
}
```

```
// Implementing double linked list

#include <stdio.h>
#include <stdlib.h>

struct node {
    struct node * prev;
    int data;
    struct node * next;
};

struct node * head = NULL;
int count = 0;

void insertAtBegin(int x);
void insertAtEnd(int x);
void insertAtPosition(int x, int pos);
void deleteAtBegin();
void deleteAtEnd();
void deleteAtPosition(int pos);
void display();
void search(int key);

int main() {
    insertAtBegin(20);
    insertAtBegin(40);
    insertAtBegin(60);
    display();

    insertAtEnd(80);
    insertAtEnd(100);
    insertAtEnd(120);
    display();

    insertAtPosition(70, 1);
    insertAtPosition(10, 4);
    insertAtPosition(130, 8);
    display();

    search(80);
    search(105);

    deleteAtBegin();
    deleteAtBegin();
    display();

    deleteAtEnd();
    deleteAtEnd();
    display();
```

```
    deleteAtPosition(4);
    deleteAtPosition(2);
    display();

    return 0;
}

void insertAtBegin(int x) {
    struct node * new;
    new = (struct node * )malloc(sizeof(struct node));
    new->prev = NULL;
    new->data = x;
    new->next = head;
    if(head != NULL)
        head->prev = new;
    head = new;
    printf("\nInsertd %d at the beginning", x);
    count++;
}

void insertAtEnd(int x) {
    if(head == NULL)
        insertAtBegin(x);
    else {
        struct node * temp = head;
        while(temp->next != NULL)
            temp = temp->next;
        struct node * new;
        new = (struct node * )malloc(sizeof(struct node));
        new->prev = temp;
        new->data = x;
        new->next = NULL;
        temp->next = new;
        printf("\nInserted %d at the end", x);
        count++;
    }
}

void insertAtPosition(int x, int pos) {
    if(pos<=0 || pos>(count+1)) {
        printf("\nInvalid position, insertion not possible");
        return;
    }
    if(pos == 1)
        insertAtBegin(x);
    else if(pos == (count+1))
        insertAtEnd(x);
    else {
        struct node * temp = head;
        for(int i=1; i<(pos-1); i++)
            temp = temp->next;
        struct node * new;
        new = (struct node * )malloc(sizeof(struct node));
        new->prev = temp;
        new->data = x;
        new->next = temp->next;
        temp->next->prev = new;
        temp->next = new;
        printf("\nInserted %d at position %d", x, pos);
        count++;
    }
}

void deleteAtBegin() {
    if(head == NULL) {
        printf("\nDouble linked list is empty");
        return;
    }

    struct node * temp = head;
    head = head->next;
    if(head != NULL)
        head->prev = NULL;
    printf("\nDeleted %d from the beginning", temp->data);
    free(temp);
    count--;
}

void deleteAtEnd() {
    if(head == NULL) {
        printf("\nDouble linked list is empty");
        return;
    }

    if(head->next == NULL)
        deleteAtBegin();
    else {
        struct node * temp = head;
        while(temp->next != NULL)
            temp = temp->next;
        temp->prev->next = NULL;
        printf("\nDeleted %d from the end", temp->data);
        free(temp);
        count--;
    }
}

void deleteAtPosition(int pos) {
    if(head == NULL) {
```

```
        printf("\nDouble linked list is empty");
        return;
    }

    if(pos<=0 || pos>count) {
        printf("\nInvalid position, deletion not possible");
        return;
    }
    if(pos == 1)
        deleteAtBegin();
    else if(pos == count)
        deleteAtEnd();
    else {
        struct node * temp = head;
        for(int i=1; i<pos; i++)
            temp = temp->next;
        temp->prev->next = temp->next;
        temp->next->prev = temp->prev;
        printf("\nDeleted %d from position %d", temp->data, pos);
        free(temp);
        count--;
    }
}

void display() {
    if(head == NULL) {
        printf("\nNo elements in the double linked list");
        return;
    }
    struct node * start = head;
    struct node * end;
    printf("\nElements in the double linked list are: ");
    printf("\nForward traversing: \n");
    printf("head --> ");
    while(start != NULL) {
        printf("%d --> ", start->data);
        end = start;
        start = start->next;
    }
    printf("tail");
    printf("\nBackward traversing: \n");
    printf("tail --> ");
    while(end != NULL) {
        printf("%d --> ", end->data);
        end = end->prev;
    }
    printf("head");
    printf("\n");
}
```

```
void search(int key) {
    if(head == NULL) {
        printf("\nNo elements in the double linked list");
        return;
    }

    struct node * temp = head;
    int flag=1, pos=1;
    printf("\nSearching %d: \n", key);
    while(temp != NULL) {
        if(temp->data == key) {
            printf("Found at position: %d", pos);
            flag=0;
        }
        temp = temp->next;
        pos++;
    }
    if(flag)
        printf("Key not found");
    printf("\n");
}
```

```
// Implementing circular linked list

#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node * link;
};

struct node * head = NULL;
int count = 0;

void insertAtBegin(int x);
void insertAtEnd(int x);
void insertAtPosition(int x, int pos);
void deleteAtBegin();
void deleteAtEnd();
void deleteAtPosition(int pos);
void display();
void search(int key);

int main() {
    insertAtBegin(70);
    insertAtBegin(120);
    insertAtBegin(90);
    display();

    insertAtEnd(60);
    insertAtEnd(45);
    insertAtEnd(105);
    display();

    insertAtPosition(30, 2);
    insertAtPosition(85, 5);
    insertAtPosition(50, 7);
    display();

    search(90);
    search(115);

    deleteAtBegin();
    deleteAtBegin();
    display();

    deleteAtEnd();
    deleteAtEnd();
    display();

    deleteAtPosition(4);
    deleteAtPosition(2);
    display();

    return 0;
}

void insertAtBegin(int x) {
    struct node * new;
    new = (struct node * )malloc(sizeof(struct node));
    new->data = x;
    new->link = head;
    head = new;
    if(new->link == NULL)
        new->link = head;
    else {
        struct node * temp = new->link;
        while(temp->link != new->link)
            temp = temp->link;
        temp->link = head;
    }
    printf("\nInserted %d at the beginnig", x);
    count++;
}

void insertAtEnd(int x) {
    if(head == NULL)
        insertAtBegin(x);
    else {
        struct node * temp = head;
        while(temp->link != head)
            temp = temp->link;
        struct node * new;
        new = (struct node * )malloc(sizeof(struct node));
        new->data = x;
        new->link = head;
        temp->link = new;
        printf("\nInserted %d at the end", x);
        count++;
    }
}

void insertAtPosition(int x, int pos) {
    if(pos<=0 || pos>(count+1)) {
        printf("\nInvalid position, insertion not possible");
        return;
    }
    if(pos == 1)
        insertAtBegin(x);
    else if(pos == (count+1))
```

```
        insertAtEnd(x);
    else {
        struct node * temp = head;
        for(int i=1; i<(pos-1); i++)
            temp = temp->link;
        struct node * new;
        new = (struct node * )malloc(sizeof(struct node));
        new->data = x;
        new->link = temp->link;
        temp->link = new;
        printf("\nInserted %d at position %d", x, pos);
        count++;
    }
}

void deleteAtBegin() {
    if(head == NULL) {
        printf("\nCircular linked list is empty");
        return;
    }

    struct node * temp = head;
    if(head->link == head)
        head = NULL;
    else {
        while(temp->link != head)
            temp = temp->link;
        temp->link = head->link;
        temp = head;
        head = head->link;
    }
    printf("\nDeleted %d from the beginning", temp->data);
    free(temp);
    count--;
}

void deleteAtEnd() {
    if(head == NULL) {
        printf("\nCircular linked list is empty");
        return;
    }

    if(head->link == head)
        deleteAtBegin();
    else {
        struct node * prev = head;
        struct node * temp = head->link;
        while(temp->link != head) {
            temp = temp->link;
            prev = prev->link;
        }
        prev->link = head;
        printf("\nDeleted %d from the end", temp->data);
        free(temp);
        count--;
    }
}

void deleteAtPosition(int pos) {
    if(head == NULL) {
        printf("\nCircular linked list is empty");
        return;
    }

    if(pos<=0 || pos>count) {
        printf("\nInvalid position, deletion not possible");
        return;
    }
    if(pos == 1)
        deleteAtBegin();
    else if(pos == count)
        deleteAtEnd();
    else if(pos>1 && pos<count) {
        struct node * prev = head;
        struct node * temp = head->link;
        for(int i=2; i<pos; i++) {
            temp = temp->link;
            prev = prev->link;
        }
        prev->link = temp->link;
        printf("\nDeleted %d from position %d", temp->data, pos);
        free(temp);
        count--;
    }
}

void display() {
    if(head == NULL) {
        printf("\nNo elements in the circular linked list");
        return;
    }

    struct node * temp = head;
    printf("\nElements in the circular linked list are: \n");
    printf("head --> ");
    do {
        printf("%d --> ", temp->data);
        temp = temp->link;
    } while(temp != head);
    printf("head");
```

```
    printf("\n");
}

void search(int key) {
    if(head == NULL) {
        printf("\nNo elements in the circular linked list\n");
        return;
    }

    struct node * temp = head;
    int flag=1, pos=1;
    printf("\nSearching %d: \n", key);
    do {
        if(temp->data == key) {
            printf("Found at position: %d", pos);
            flag = 0;
        }
        temp = temp->link;
        pos++;
    } while(temp != head);
    if(flag)
        printf("Key not found");
    printf("\n");
}
```